

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



TRAN TRUNG DUNG – 523H0130
TRUONG NGUYEN BAO – 522H0166

MIDTERM REPORT
DEEP LEARNING

HO CHI MINH CITY, 2025

VIETNAM GENERAL CONFEDERATION OF LABOR

TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



TRAN TRUNG DUNG – 523H0130
TRUONG NGUYEN BAO – 522h0166

MIDTERM REPORT

DEEP LEARNING

Advised by

M.A. LE ANH CUONG

HO CHI MINH CITY, 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to M.A. Le Anh Cuong, our instructor and mentor, for his valuable guidance and support throughout the midterm report. He has been very helpful and patient in providing us with constructive feedback and suggestions to improve our work. He has also encouraged us to explore new technologies and techniques to enhance our system's functionality and performance. We have learned a lot from his expertise and experience in Deep learning. We are honored and privileged to have him as our teacher and supervisor.

Ho Chi Minh city, 23 March 2025.

Author

(Signature and full name)

Dung

Tran Trung Dung

Bao

Truong Nguyen Bao

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by M.A. Le Anh Cuong; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, We will take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 23 March 2025

Author

(Signature and full name)

Dung

Tran Trung Dung

Bao

Truong Nguyen Bao

Contents

CHAPTER 1. Tìm hiểu lý thuyết về bài toán VQA (Visual Question Answering): 2

1.1) Khái niệm và các đặc điểm của VQA (Visual Question Answering):2

1.1.1) Khái niệm của VQA:.....2

1.1.2) Các nhiệm vụ của VQA:3

1.1.3) Các yếu tố trong VQA:.....6

1.2) Mô tả mô hình và các hướng tiếp cận chính trong bài toán VQA7

1.2.1) Tổng quan về bài toán VQA7

1.2.2) Kiến trúc mô hình VQA tổng quát7

1.2.3) Các cách tiếp cận của mô hình VQA8

REFERENCES..... 34

CHAPTER 1. Tìm hiểu lý thuyết về bài toán VQA (Visual Question Answering):

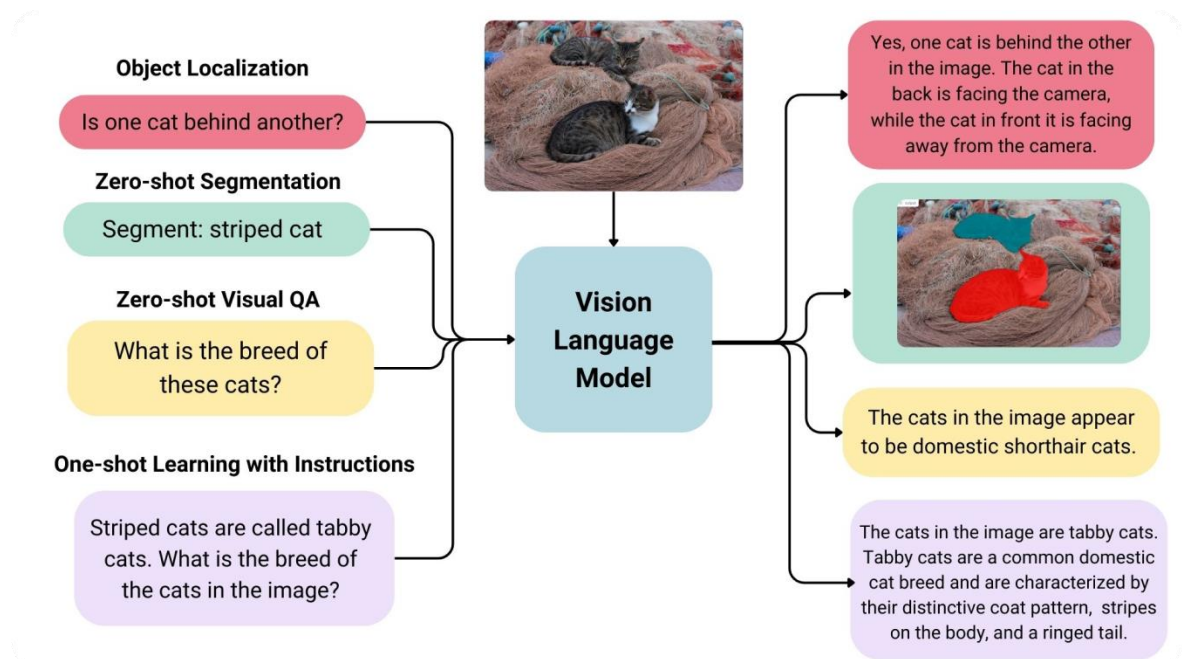
1.1) Khái niệm và các đặc điểm của VQA (Visual Question Answering):

1.1.1) Khái niệm của VQA:

Visual Question Answering (VQA) là một lĩnh vực trong trí tuệ nhân tạo (**AI**), nơi hệ thống máy tính được yêu cầu trả lời các câu hỏi liên quan đến một hình ảnh. VQA kết hợp giữa thị giác máy tính (**computer vision**) và xử lý ngôn ngữ tự nhiên (**natural language processing**), với mục tiêu giúp máy tính hiểu và phân tích thông tin từ cả hình ảnh và ngữ nghĩa của câu hỏi.

Ngoài ra, **Visual Question Answering (VQA)** cũng là một bài toán thú vị mô phỏng gần nhất khả năng của con người. Nói một cách ngắn gọn, ta cần huấn luyện máy để có thể thực hiện tương tác hỏi đáp giữa người và máy về một hình ảnh trực quan. Hiện tại, bài toán này chủ yếu thực hiện trên tiếng Anh vì có sẵn các VQA benchmark.

Cụ thể, trong VQA, hệ thống nhận vào một hình ảnh và một câu hỏi dạng ngữ nghĩa (ví dụ: "Có bao nhiêu người trong bức tranh?" hoặc "Con mèo đang làm gì?") và trả về câu trả lời phù hợp (chẳng hạn như "3" hoặc "Đang ngủ").



Hình 1.1: Ví dụ về khái niệm của 1 model VQA

1.1.2) Các nhiệm vụ của VQA:

Trong bài toán **Visual Question Answering (VQA)**, để trả lời chính xác các câu hỏi về hình ảnh, hệ thống cần thực hiện một số **nhiệm vụ chính**. Những nhiệm vụ này liên quan đến việc xử lý cả **hình ảnh** và **câu hỏi** để đưa ra câu trả lời hợp lý. Dưới đây là các nhiệm vụ quan trọng trong VQA:

Bước 1: Nhận diện đối tượng trong hình ảnh (Object Detection):

Đây là bước đầu tiên quan trọng, nơi hệ thống cần phải **xác định các đối tượng có mặt trong hình ảnh**. Đối tượng có thể là người, động vật, đồ vật, hoặc các cảnh quan khác. Hệ thống cần phân biệt được các đối tượng này để trả lời các câu hỏi liên quan.

- **Ví dụ:** Nếu câu hỏi là "Có bao nhiêu chiếc ô tô trong bức tranh?", hệ thống phải có khả năng nhận diện ô tô và đếm số lượng chúng trong hình ảnh.
- **Các kỹ thuật sử dụng:**
 - **Convolutional Neural Networks (CNNs):** Dùng để nhận diện đối tượng và trích xuất đặc trưng từ hình ảnh.
 - **Region-based CNN (R-CNN):** Để phát hiện và phân vùng đối tượng trong hình ảnh.

Bước 2: Phân tích mối quan hệ giữa các đối tượng (Relationship Understanding)

Khi có nhiều đối tượng trong hình ảnh, hệ thống cần hiểu **mối quan hệ giữa các đối tượng đó**. Điều này rất quan trọng đối với các câu hỏi yêu cầu hiểu sự tương tác hoặc vị trí của các đối tượng trong hình ảnh.

- **Ví dụ:** "Con mèo đang ngồi bên cạnh chiếc bàn" yêu cầu hệ thống hiểu mối quan hệ không chỉ giữa con mèo và chiếc bàn, mà còn xác định đúng vị trí và hành động của con mèo.

Các kỹ thuật sử dụng:

- **Graph Neural Networks (GNNs):** Dùng để mô hình hóa mối quan hệ giữa các đối tượng trong hình ảnh dưới dạng đồ thị.
- **Spatial Attention:** Chú trọng đến không gian và vị trí của các đối tượng trong hình ảnh.

Bước 3: Hiểu ngữ nghĩa câu hỏi (Question Understanding)

Để trả lời câu hỏi chính xác, hệ thống phải **hiểu ngữ nghĩa câu hỏi**. Điều này bao gồm việc phân tích câu hỏi để xác định yêu cầu của nó. Ví dụ, câu hỏi có thể yêu cầu một con số (ví dụ: "Có bao nhiêu người?") hoặc mô tả một hành động (ví dụ: "Con chó đang làm gì?").

- **Ví dụ:** Để trả lời câu hỏi "Có bao nhiêu chiếc ô tô?", hệ thống cần nhận ra rằng đây là một câu hỏi yêu cầu số lượng.

Các kỹ thuật sử dụng:

- **Natural Language Processing (NLP):** Các mô hình như RNN, LSTM hoặc Transformer được sử dụng để phân tích câu hỏi và hiểu các phân tử ngữ nghĩa trong câu hỏi.
- **Question Encoding:** Mã hóa câu hỏi thành các vector hoặc biểu diễn số học để hệ thống có thể "hiểu" câu hỏi một cách chính xác.

Bước 4: Tích hợp thông tin từ hình ảnh và câu hỏi (Multimodal Fusion)

Một trong những nhiệm vụ quan trọng nhất trong VQA là kết hợp thông tin từ **hình ảnh** và **câu hỏi** để đưa ra câu trả lời. Phần này thường được gọi là "multimodal

fusion", nơi hệ thống phải phối hợp giữa thông tin thị giác từ hình ảnh và thông tin ngữ nghĩa từ câu hỏi.

- **Ví dụ:** Để trả lời câu hỏi "Có bao nhiêu quả bóng trong bức tranh?", hệ thống cần kết hợp thông tin về hình ảnh (những quả bóng xuất hiện) và câu hỏi (yêu cầu số lượng quả bóng).

Các kỹ thuật sử dụng:

- **Fusion Networks:** Mô hình kết hợp thông tin giữa thị giác và ngôn ngữ, ví dụ như kết hợp các đặc trưng hình ảnh (từ CNN) và các đặc trưng ngôn ngữ (từ RNN hoặc Transformer).
- **Attention Mechanisms:** Sử dụng các cơ chế chú ý để lựa chọn những vùng quan trọng trong hình ảnh dựa trên câu hỏi và ngược lại.

Bước 5: Tạo ra câu trả lời (Answer Generation)

Sau khi đã xử lý và hiểu cả hình ảnh lẫn câu hỏi, hệ thống cần **tạo ra câu trả lời** chính xác. Câu trả lời có thể là một từ, một cụm từ hoặc thậm chí một câu dài tùy vào yêu cầu của câu hỏi.

- **Ví dụ:** Nếu câu hỏi là "Có bao nhiêu chiếc xe?", câu trả lời sẽ là một số (ví dụ: "2").

Các kỹ thuật sử dụng:

- **Classification:** Nếu câu trả lời là một từ có sẵn trong bộ dữ liệu (như "Có" hoặc "Không"), mô hình có thể sử dụng phân loại để đưa ra câu trả lời.
- **Generation Models:** Nếu câu trả lời yêu cầu tạo ra văn bản, mô hình có thể sử dụng các mạng sinh (GANs hoặc Transformer) để sinh ra câu trả lời.

Bước 6: Xử lý các câu hỏi phức tạp (Handling Complex Queries)

Đây là nhiệm vụ giải quyết các câu hỏi phức tạp, chẳng hạn như những câu hỏi yêu cầu suy luận logic hoặc kết hợp nhiều thông tin từ hình ảnh và câu hỏi.

- **Ví dụ:** "Nếu có 3 chiếc xe và 2 người, ai đang lái xe?" Đây là câu hỏi yêu cầu hệ thống suy luận không chỉ về số lượng đối tượng mà còn về hành động và mối quan hệ giữa chúng.

Các kỹ thuật sử dụng:

- **Reasoning Networks:** Các mô hình có khả năng suy luận logic như **Neural Symbolic Networks** hoặc **Relational Networks**.
- **Memory-Augmented Networks:** Các mạng này có bộ nhớ dài hạn giúp lưu trữ thông tin và suy luận qua nhiều bước.

1.1.3) Các yếu tố trong VQA:

1. **Hình ảnh (Image):** Hình ảnh có thể là bất kỳ loại nào (ảnh chụp, tranh vẽ, video) dùng để làm “input” của bài toán VQA => dựa vào input là hình ảnh để trả lời các câu hỏi liên qua về hình ảnh đó.
2. **Câu hỏi (Question):** Câu hỏi cũng là 1 phần của input, câu hỏi có thể yêu cầu thông tin cụ thể về hình ảnh, chẳng hạn như thông tin về đối tượng, mối quan hệ giữa các đối tượng, hoặc hành động diễn ra trong hình ảnh.
3. **Câu trả lời (Answer):** Câu trả lời có thể hiểu đơn giản chính là kết quả output mà mô hình sẽ trả ra cho người dùng. có thể là một câu đơn giản (ví dụ: "Có", "Không") hoặc một câu dài hơn (ví dụ: tên đối tượng, số lượng đối tượng).

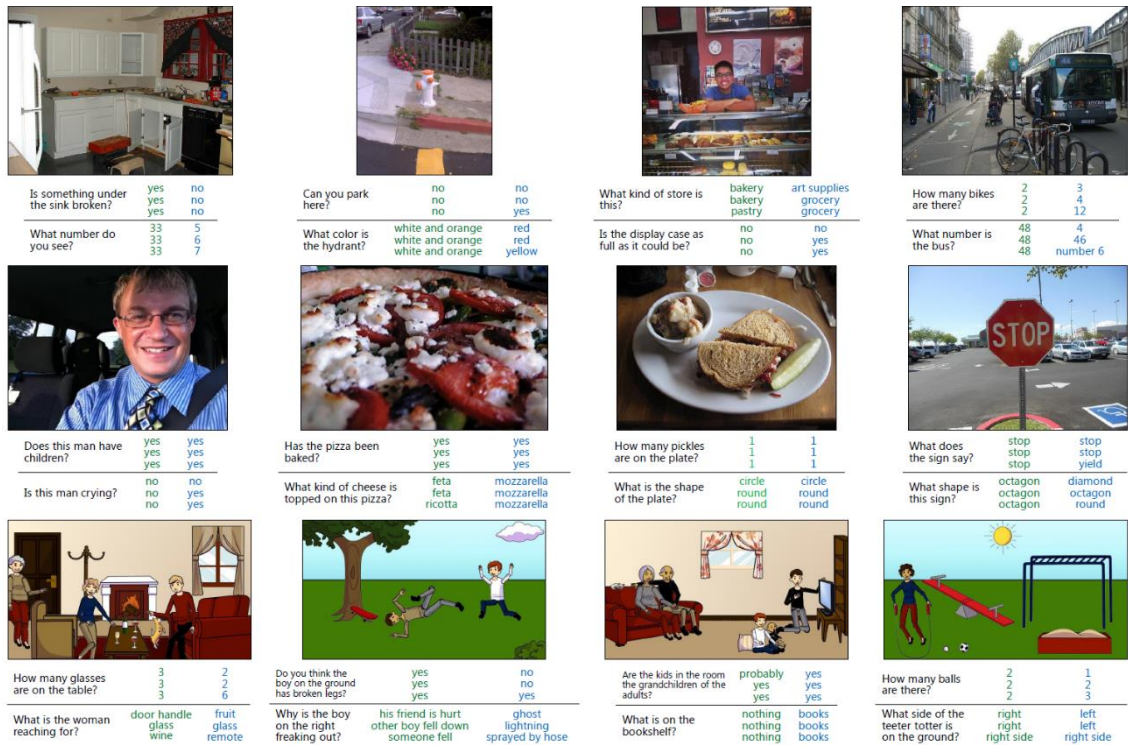


Fig. 2: Examples of questions (black), (a subset of the) answers given when looking at the image (green), and answers given when not looking at the image (blue) for numerous representative examples of the dataset. See the appendix for more examples.

Hình 1.2: Những ví dụ về các trường hợp của bài toán VQA

1.2) Mô tả mô hình và các hướng tiếp cận chính trong bài toán VQA

1.2.1) Tổng quan về bài toán VQA

Visual Question Answering (VQA) là một bài toán trong lĩnh vực Trí tuệ nhân tạo kết hợp giữa **Thị giác máy tính (Computer Vision)** và **Xử lý ngôn ngữ tự nhiên (Natural Language Processing)**. Mục tiêu chính của VQA là xây dựng một hệ thống có khả năng **trả lời câu hỏi dưới dạng văn bản dựa trên nội dung của một hình ảnh**.

1.2.2) Kiến trúc mô hình VQA tổng quát

Một mô hình VQA hiện đại thường gồm 4 thành phần chính:

1. Trích xuất đặc trưng từ hình ảnh (Image Feature Extraction)

2. **Mã hóa ngữ nghĩa câu hỏi** (Question Encoding)
3. **Kết hợp thông tin đa phương thức** (Multimodal Fusion)
4. **Sinh hoặc chọn câu trả lời** (Answer Prediction or Generation)

1.2.3) Các cách tiếp cận của mô hình VQA

1.Feature Fusion (Kết hợp các đặc trưng):

Feature Fusion là gì ?

Feature fusion (kết hợp đặc trưng) là quá trình kết hợp thông tin đặc trưng từ hai nguồn dữ liệu khác nhau, trong trường hợp bài toán VQA:

- **Đặc trưng hình ảnh** (trích xuất từ hình ảnh bằng CNN hoặc các mô hình thị giác),
- **Đặc trưng ngôn ngữ** (mã hóa câu hỏi bằng LSTM, Transformer, v.v.).

⇒ **Biểu diễn chung (joint representation)**, nơi cả hai loại thông tin này được tích hợp để mô hình có thể hiểu mối liên hệ giữa ảnh và câu hỏi, từ đó đưa ra câu trả lời chính xác.

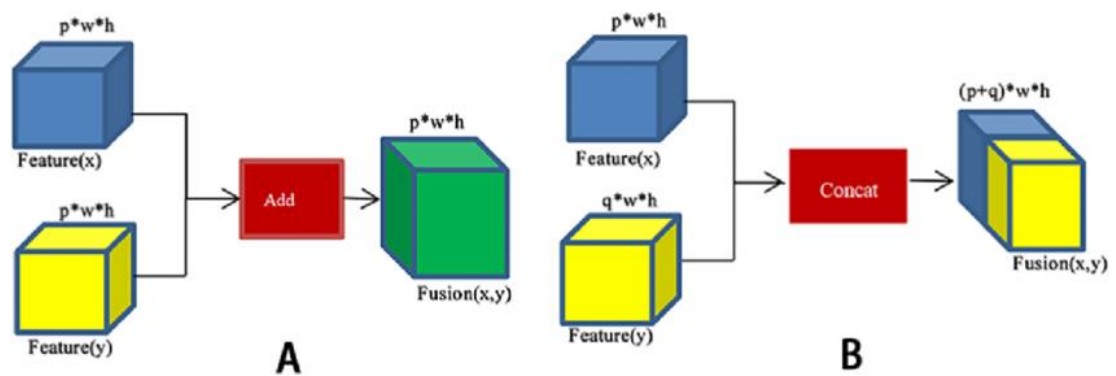
- Tại sao cần Feature Fusion?

- Hình ảnh và câu hỏi là hai loại dữ liệu khác biệt (ảnh là dữ liệu thị giác, câu hỏi là dữ liệu ngôn ngữ).
- Mỗi loại dữ liệu được biểu diễn dưới dạng vector đặc trưng riêng biệt.
- Để hệ thống VQA có thể xử lý và trả lời, nó cần **kết hợp hai vector đặc trưng này thành một vector duy nhất** hoặc một biểu diễn thống nhất.

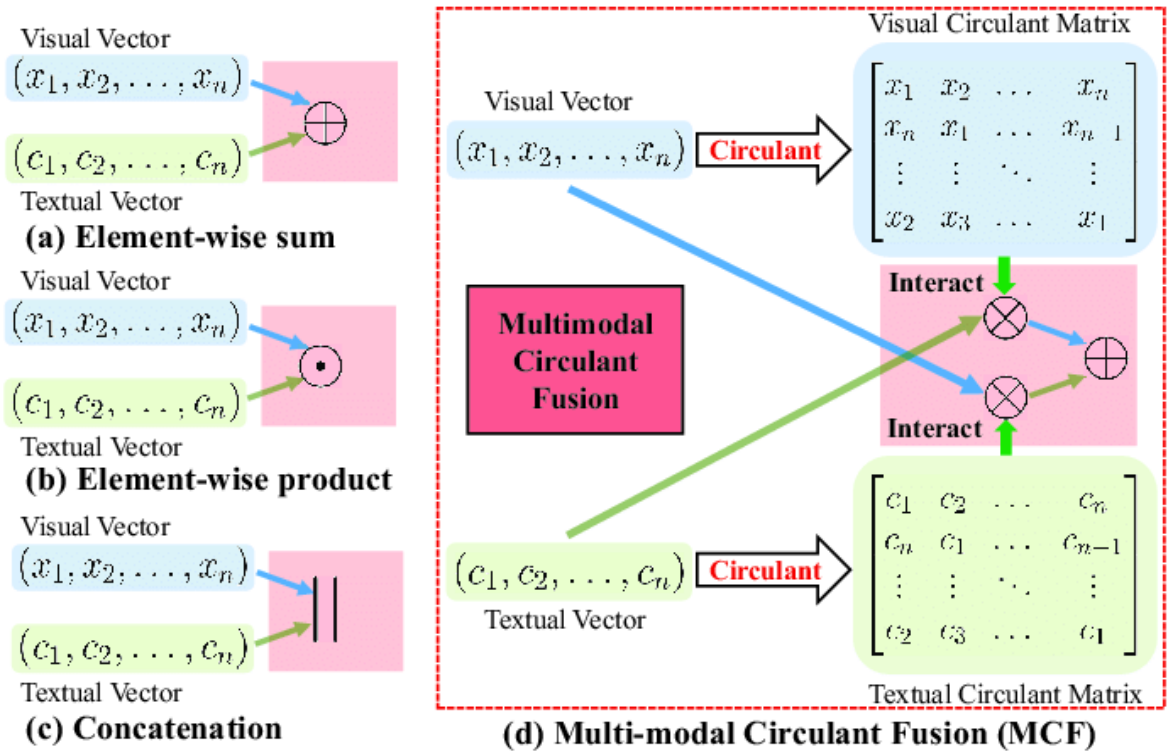
- Các phương pháp phổ biến để Feature Fusion

- **Concatenation** (Ghép nối): Ghép hai vector đặc trưng từ ảnh và câu hỏi lại thành một vector dài hơn.
- **Element-wise Product hoặc Sum (Phép nhân/cộng từng phần tử)**: Thực hiện phép nhân hoặc cộng từng phần tử tương ứng giữa hai vector.

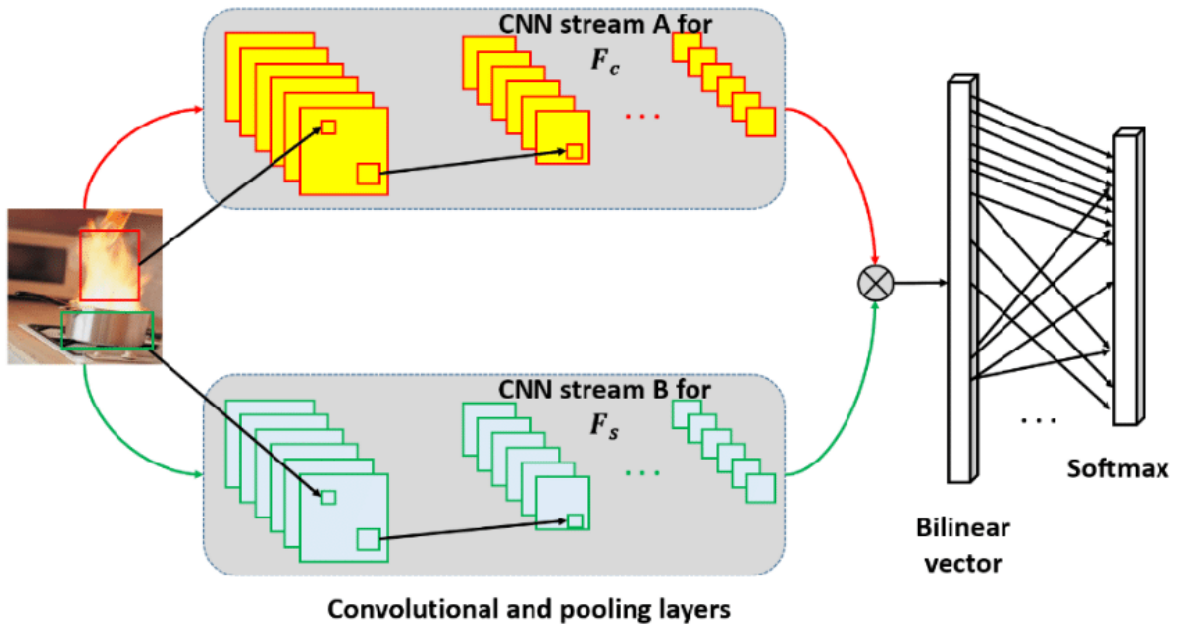
- **Bilinear Pooling** (Phép kết hợp Bilinear): Phương pháp phức tạp hơn, dùng phép nhân ma trận để kết hợp đặc trưng, ví dụ: MCB (Multimodal Compact Bilinear pooling), MLB (Multimodal Low-rank Bilinear pooling), MUTAN (Multimodal Tucker Fusion)
- **Multimodal Transformer**: Sử dụng kiến trúc Transformer để kết hợp đặc trưng từ cả ảnh và ngôn ngữ trong cùng một mô hình.



Hình 1.3: Các phương pháp kết hợp đặc trưng (Feature Fusion) trong mô hình VQA: (A) phép cộng, (B) phép nối (concatenation)



Hình 1.4: Các phương pháp kết hợp đặc trưng (feature fusion) trong mô hình VQA. Bao gồm (a) Element-wise sum, (b) Element-wise product, (c) Concatenation, (d) Multi-modal Circulant Fusion (MCF)



Hình 1.5: Phương pháp Bilinear pooling trong việc kết hợp đặc trưng (feature fusion)

Phương pháp	Cách hoạt động	Ưu điểm	Nhược điểm
Concatenation	Ghép vector lại	Đơn giản, dễ dùng	Không nắm tốt quan hệ
Element-wise product/sum	Nhân hoặc cộng từng phần tử	Tăng tương tác	Có thể mất thông tin
Bilinear pooling	Nhân ma trận, mô hình phi tuyến tính	Mô hình quan hệ phức tạp	Tính toán phức tạp
Multimodal Transformer	Transformer kết hợp đặc trưng đa phương thức	Hiệu quả, hiện đại	Cần nhiều dữ liệu

Bảng tổng hợp các phương pháp phổ biến của Feature fusion trong mô hình VQA

2.Attention Mechanism trong VQA

Attention Mechanism là gì ?

Khái niệm: Attention Mechanism là một kỹ thuật trong deep learning giúp mô hình tập trung vào những phần quan trọng nhất của dữ liệu đầu vào khi đưa ra dự đoán. Trong bài toán Visual Question Answering (VQA), attention đặc biệt hữu ích để mô hình chỉ tập trung vào vùng ảnh có liên quan đến câu hỏi.

Nguyên lý hoạt động:

- Một **chuỗi input** (câu hỏi hoặc các patch ảnh).
- Một **context** cần xử lý (ví dụ: câu hỏi để định hướng chú ý lên ảnh).

Attention tính toán trọng số (**weights**) cho từng phần tử trong input, thể hiện mức độ quan trọng của nó đối với ngữ cảnh. Kết quả là mô hình có thể tập trung vào thông tin liên quan và bỏ qua nhiễu.

Công thức cơ bản (scaled dot-product attention trong Transformer):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Trong đó:

- **Q (Query):** đặc trưng cần tìm thông tin.
- **K (Key):** đặc trưng dùng để so sánh.
- **V (Value):** đặc trưng được tổng hợp.
- Kết quả: vector tổng hợp có trọng số.

Các loại Attention trong VQA

- **Visual Attention**

- Mục tiêu: giúp mô hình tập trung vào những **vùng ảnh quan trọng**, có liên quan trực tiếp đến câu hỏi thay vì toàn bộ ảnh.
- Ví dụ: khi hỏi “*What animal is in the picture?*”, mô hình sẽ tập trung vào vùng có **con bò** trong ảnh thay vì cả cánh đồng.

- **Question Attention**

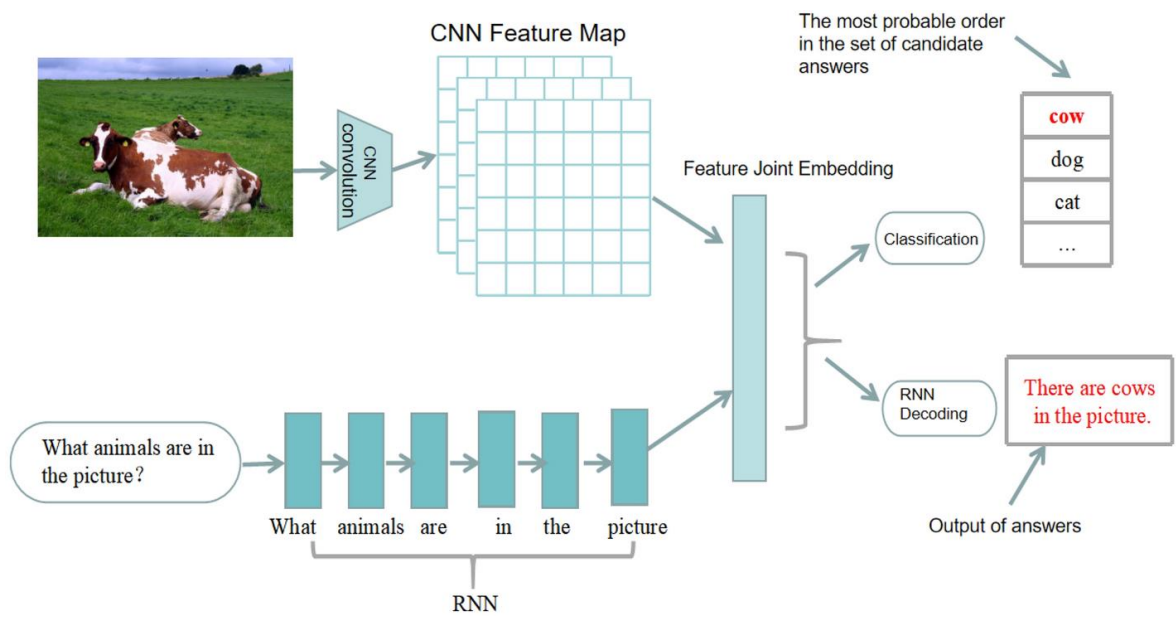
- Mục tiêu: xác định **từ hoặc cụm từ quan trọng** trong câu hỏi để hướng dẫn mô hình tìm thông tin phù hợp trong ảnh.
- Ví dụ: trong câu hỏi “*What color is the car?*”, mô hình sẽ tập trung mạnh vào các từ “**color**” và “**car**”.

- **Co-Attention**

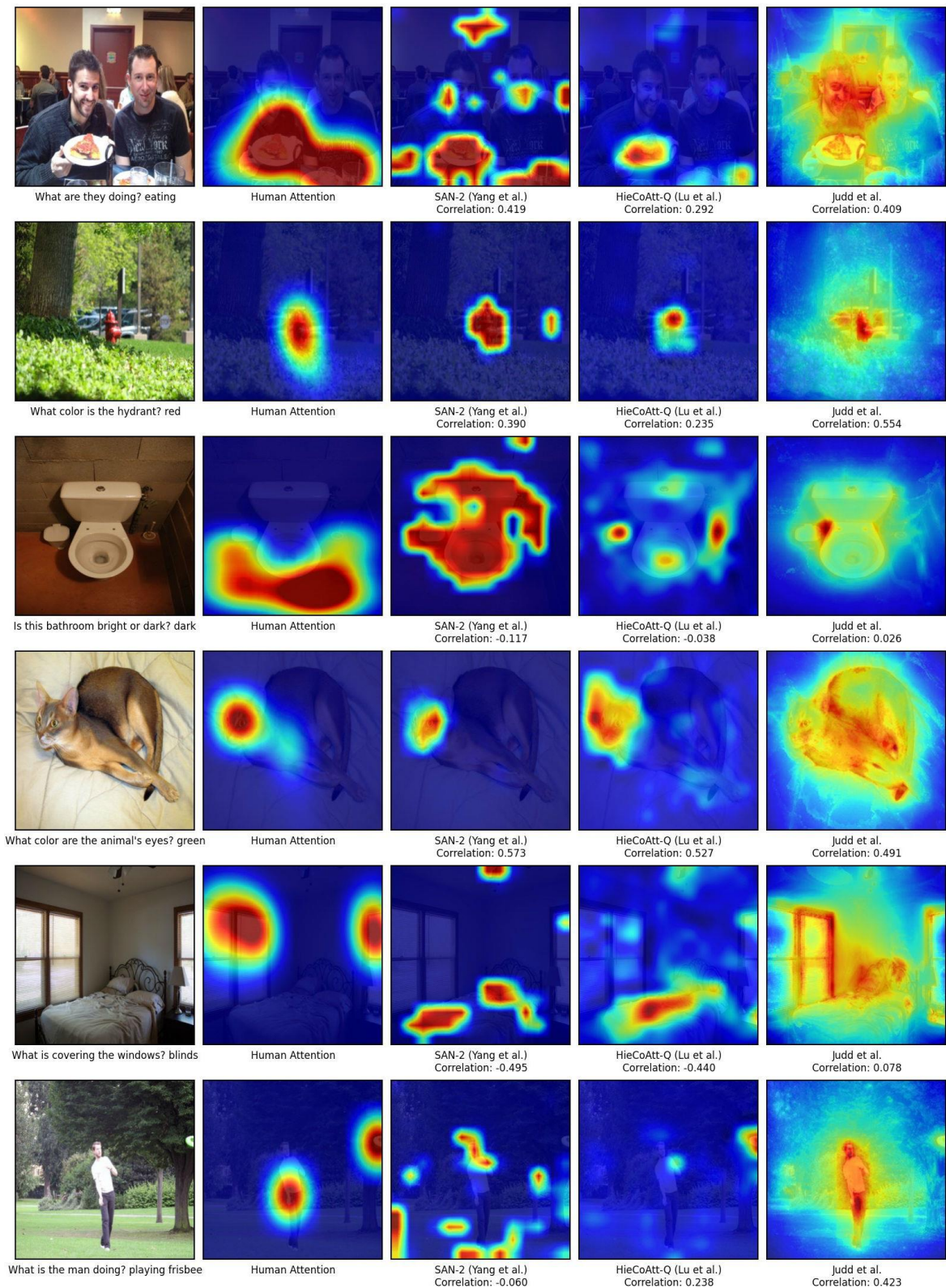
- Mục tiêu: cho phép **ảnh và câu hỏi chú ý lẫn nhau**. Ảnh định hướng việc chọn từ trong câu hỏi, đồng thời câu hỏi định hướng việc chọn vùng ảnh quan trọng.
- Cách này tạo ra sự **tương tác song phương**, giúp mô hình hiểu ngữ cảnh tốt hơn.

- **Cross-Modal Attention**

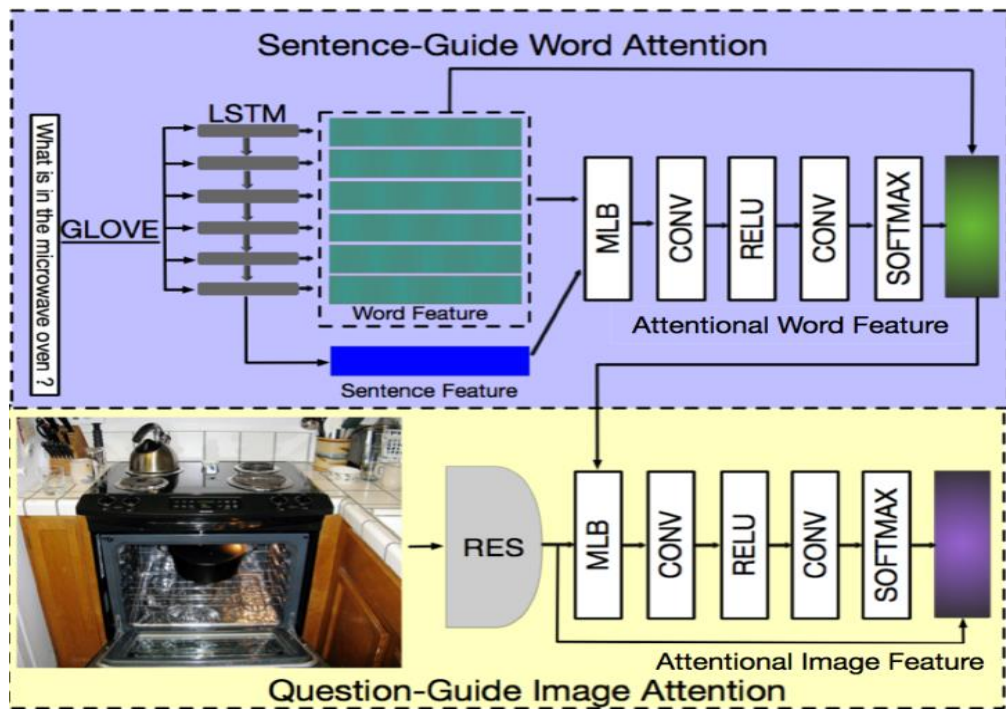
- Mục tiêu: sử dụng cơ chế **multi-head attention** để học mối quan hệ phức tạp giữa đặc trưng ảnh và văn bản.
- Cách này cho phép hai modality (ảnh và câu hỏi) trao đổi thông tin sâu hơn, thường được dùng trong các mô hình VQA hiện đại như **ViLBERT**, **LXMERT**, **BLIP**.



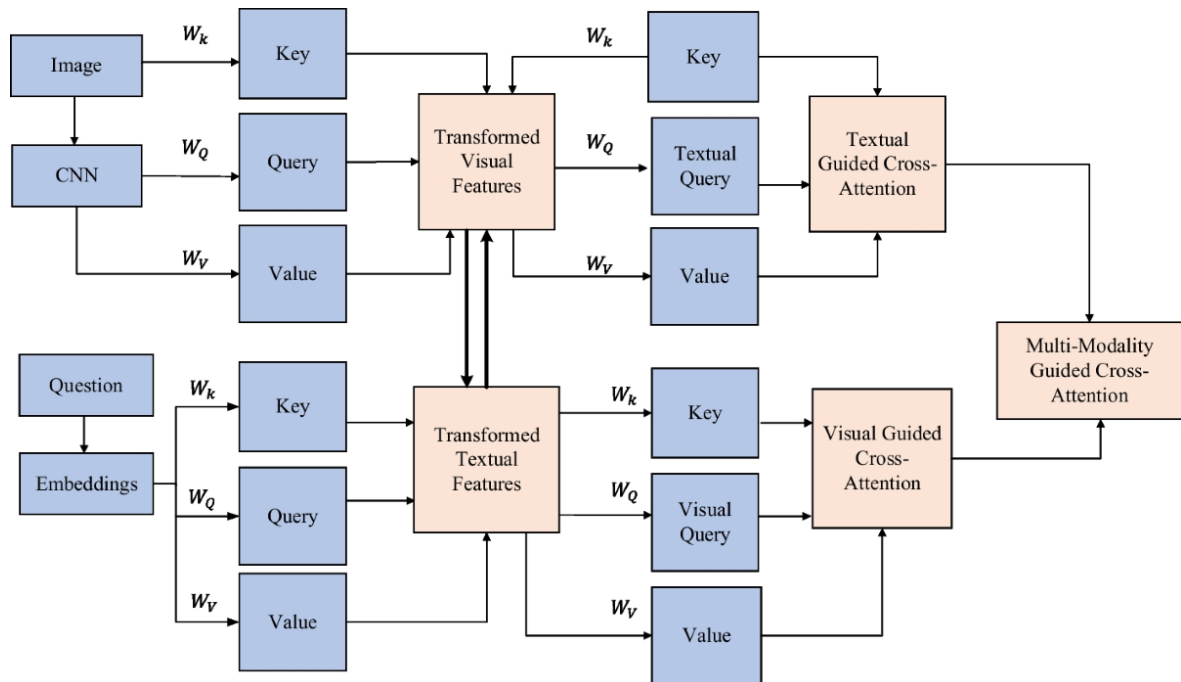
Hình 1.6: Visual Attention xác định vùng ảnh quan trọng (con bò) bằng heatmap để hỗ trợ trả lời câu hỏi



Hình 1.7: Question Attention: xác định từ/cụm từ quan trọng trong câu hỏi để hướng dẫn mô hình tập trung



Hình 1.8: Co-Attention cùng tính toán trọng số chú ý giữa ảnh và câu hỏi ảnh hướng dẫn câu hỏi và câu hỏi hướng dẫn ảnh để tăng tương tác song phương



Hình 1.10: Cross-Modal Attention: sử dụng ảnh làm truy vấn (query) và văn bản làm key-value để tính trọng số chú ý giữa hai modality giúp mô hình định hướng thông tin ảnh - câu hỏi

2. Pre-trained models

- Khái niệm:

Pre-trained model là một mô hình đã được huấn luyện trước trên một tập dữ liệu rất lớn, thường bởi các tổ chức hoặc chuyên gia, và sau đó được chia sẻ để người khác **tái sử dụng** cho các tác vụ tương tự hoặc có liên quan.

Ví dụ:

- Trong thị giác máy tính (Computer Vision): các mô hình như **ResNet**, **VGG**, **Inception**, **ViT** được huấn luyện trước trên tập **ImageNet** (chứa hàng triệu hình ảnh).
- Trong xử lý ngôn ngữ tự nhiên (NLP): các mô hình như **BERT**, **GPT**, **T5**, **RoBERTa** được huấn luyện trước trên hàng tỷ từ từ Internet.

- **Mục đích:**

- Giúp **tiết kiệm thời gian và tài nguyên**, vì bạn **không cần huấn luyện từ đầu (from scratch)**.
- Giúp mô hình **học nhanh hơn và đạt hiệu suất cao hơn** trên tác vụ mới, nhờ đã có kiến thức tổng quát từ tập dữ liệu lớn trước đó.

- **Cách sử dụng:**

	Mô tả	Ví dụ
Feature extraction	Dùng phần trích đặc trưng của mô hình pre-trained, đóng băng (freeze) trọng số, chỉ huấn luyện phần cuối (classifier).	Dùng ResNet làm bộ trích đặc trưng ảnh cho bài toán phân loại mới.
Fine-tuning	Giữ nguyên kiến trúc mô hình, huấn luyện lại một phần hoặc toàn bộ trọng số trên dữ liệu của bạn để tinh chỉnh.	Fine-tune BERT trên dữ liệu hội thoại tiếng Việt.

- **Ưu điểm:**

- Tiết kiệm tài nguyên tính toán
- Dễ áp dụng cho các bài toán mới
- Hiệu quả cao với lượng dữ liệu nhỏ
- Học được đặc trưng tổng quát (ví dụ: hình dạng, ngữ pháp, ngữ nghĩa, v.v.)

- **Nhược điểm:**

- Có thể **thiếu tính đặc thù** nếu dữ liệu mới khác xa tập gốc
- **Fine-tune sai cách** dễ gây *catastrophic forgetting* (mất kiến thức cũ)
- **Dung lượng lớn**, khó triển khai trên thiết bị yếu

CHAPTER 2. Thực Nghiệm

2.1) Chọn dataset VQA trong một cuộc thi VQA để thực nghiệm

2.1.1 Giới thiệu về tập dữ liệu

Trong quá trình thực nghiệm, tôi lựa chọn sử dụng **tập dữ liệu Visual Question Answering (VQA)** được công bố trên nền tảng [Kaggle](#) bởi tác giả *Bhavik Ardeshta*.

Tập dữ liệu này được thiết kế để phục vụ cho các bài toán kết hợp giữa **thị giác máy tính (Computer Vision)** và **xử lý ngôn ngữ tự nhiên (Natural Language Processing)** — cụ thể là **Visual Question Answering (VQA)**, một nhiệm vụ trong đó mô hình cần **trả lời câu hỏi bằng ngôn ngữ tự nhiên dựa trên nội dung của hình ảnh**.

2.1.2 Nguồn gốc và đặc điểm của dataset

Dataset này được tổng hợp từ **bộ dữ liệu COCO (Common Objects in Context)** – một bộ dữ liệu phổ biến trong lĩnh vực thị giác máy tính.

Mỗi mẫu dữ liệu trong tập đều bao gồm **một hình ảnh**, **một câu hỏi** liên quan đến hình ảnh đó, và **một câu trả lời ngắn** (thường chỉ 1 từ hoặc cụm từ).

2.1.3 Mục tiêu và ý nghĩa của bài toán dựa trên dataset này

Mục tiêu của bài toán VQA dựa trên dataset này là **xây dựng và huấn luyện một mô hình trí tuệ nhân tạo có khả năng**:

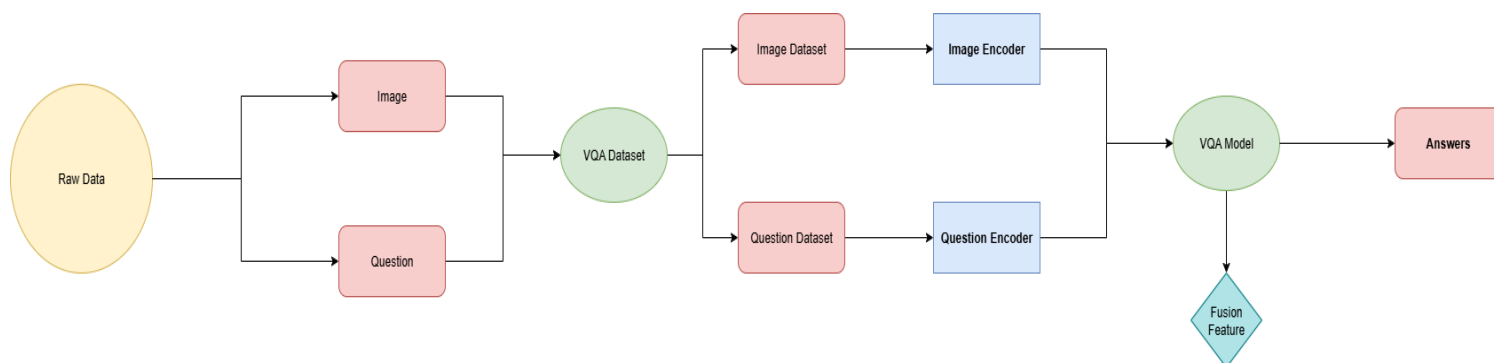
1. **Hiểu nội dung hình ảnh** (nhận diện đối tượng, cảnh vật, màu sắc, hành động, v.v.)

2. **Phân tích câu hỏi dạng ngôn ngữ tự nhiên** (hiểu từ vựng, ngữ nghĩa, cú pháp)
3. **Kết hợp hai nguồn thông tin (ảnh + câu hỏi) để sinh ra câu trả lời phù hợp nhất.**

Ví dụ:

- Ảnh: một căn phòng có cái giường ở trong
- Câu hỏi: “*What color is the bed?*”
- Mô hình cần dự đoán: “*yellow*”

2.2 Xây dựng Pipeline:



2.2.1 Trích xuất đặc trưng ảnh (Image Feature Extraction)

Trong mô hình VQA, đầu vào hình ảnh được xử lý thông qua **bộ mã hóa ảnh (Image Encoder)** nhằm trích xuất các đặc trưng thị giác cần thiết để phục vụ cho quá trình suy luận.

Cụ thể, mô hình sử dụng **mạng ResNet-50** được huấn luyện sẵn trên tập dữ liệu **ImageNet** thông qua thư viện **torchvision.models**. Lớp cuối cùng (fc) của ResNet được thay thế bằng **nn.Identity()**, giúp loại bỏ tầng phân loại gốc và chỉ giữ lại **vector đặc trưng 2048 chiều** của ảnh sau tầng trung bình toàn cục (global average pooling).

Trong quá trình huấn luyện, toàn bộ các tầng của ResNet được **đóng băng tham số (freeze)** để giữ nguyên trọng số của mô hình tiền huấn luyện, **ngoại trừ tầng “layer4”** – tầng sâu nhất – được mở khóa để cho phép tinh chỉnh (fine-tuning) trên dữ liệu VQA.

Trước khi đưa vào ResNet, hình ảnh được xử lý qua chuỗi **phép biến đổi (transform)** bao gồm:

- Thay đổi kích thước về 256 pixel (Resize(256)),
- Cắt giữa ảnh thành kích thước 224×224 (CenterCrop(224)),
- Chuyển thành tensor (ToTensor()),
- Chuẩn hóa giá trị điểm ảnh theo **mean và std của ImageNet**.

Kết quả đầu ra của giai đoạn này là một tensor đặc trưng của ảnh có kích thước **[batch_size, 2048]**, biểu diễn thông tin trực quan cô đọng của toàn bộ hình ảnh.

```
class ImageEncoder(nn.Module):
    def __init__(self, feature_dim=2048, pretrained=True):
        super().__init__()
        weights = ResNet50_Weights.DEFAULT if pretrained else ResNet50_Weights.IMAGENET1K_V1
        self.resnet = resnet50(weights=weights)

        for param in self.resnet.parameters(): #giữ nguyên các tham số để trích xuất các features nhanh tiết kiệm thời gian train
            param.requires_grad = False
        for param in self.resnet.layer4.parameters(): #mở lớp cuối cho phép cập nhật trong quá trình training giúp trích xuất các đặc trưng quan trọng.
            param.requires_grad = True
        self.resnet.fc = nn.Identity()

    def forward(self, image):
        f_img = self.resnet(image) # [batch, 2048]
        return f_img
```

2.2.2 Xử lý câu hỏi (Question Processing)

Câu hỏi đầu vào được xử lý thông qua lớp **VQADataset** và **bộ mã hóa câu hỏi (Question Encoder)** để chuyển đổi từ dạng văn bản sang biểu diễn số học có thể đưa vào mạng nơ-ron.

Trước hết, trong giai đoạn khởi tạo dataset, toàn bộ câu hỏi trong tập dữ liệu được **chuyển về chữ thường** và **token hóa** bằng công cụ nltk.word_tokenize. Từ các token này, chương trình xây dựng **từ vựng câu hỏi (question vocabulary)** giới hạn tối đa 10.000 từ, với các ký hiệu đặc biệt được gán như sau: <PAD>:0, <START>:1, <END>:2, và <UNK>:3.

Khi lấy từng mẫu dữ liệu (__getitem__), câu hỏi được mã hóa bằng cách:

1. Thêm ký hiệu <START> ở đầu và <END> ở cuối câu,

2. Chuyển từng từ sang chỉ số tương ứng trong từ điển,
3. Cắt hoặc thêm ký hiệu <PAD> để đảm bảo độ dài cố định **14 từ**.

Kết quả là một tensor kích thước **[max_len] = [14]**, chứa các chỉ số từ vựng biểu diễn cho câu hỏi.

Tiếp theo, tensor câu hỏi này được đưa vào **bộ mã hóa câu hỏi (Question Encoder)** gồm hai thành phần:

- Lớp **Embedding** ánh xạ các chỉ số từ vựng thành vector ẩn 100 chiều,
- **Mạng LSTM hai chiều (BiLSTM)** với kích thước ẩn là 512.

LSTM được thiết lập **batch_first=True** và có cơ chế **Dropout** nhằm giảm hiện tượng overfitting. Sau khi đi qua BiLSTM, hai vector ẩn cuối cùng theo hướng thuận và nghịch được **nối lại (concatenate)**, tạo thành **vector đặc trưng câu hỏi kích thước 1024 chiều**.

Kết quả của phần này là một biểu diễn ngữ nghĩa cô đọng của toàn bộ câu hỏi dưới dạng tensor có kích thước **[batch_size, 1024]**.

```
class QuestionEncoder(nn.Module):
    def __init__(self, vocab_size, embed_dim=100, hidden_dim=512, num_layers=1, dropout=0.5):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        with torch.no_grad():
            init.xavier_uniform_(self.embedding.weight) # tạo trọng số embedding bằng pp xavier_uniform
            self.embedding.weight[0] = torch.zeros(embed_dim)
            self.embedding.weight[0].requires_grad = False
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=num_layers, bidirectional=True, dropout=dropout, batch_first=True)
        self.dropout = nn.Dropout(dropout)
        self.output_dim = hidden_dim * 2 # LSTM học câu hỏi ở 2 chiều
    def forward(self, question):
        embedded = self.embedding(question)
        embedded = self.dropout(embedded)
        # LSTM
        lstm_out, (hidden, cell) = self.lstm(embedded)
        h_q = torch.cat((hidden[-2,:,:], hidden[-1,:,:]), dim=1) # (batch_size, hidden_dim*2)
        return h_q # Vector đại diện cho question
```

2.2.3 Thiết kế cơ chế kết hợp (Feature Fusion and Answer Prediction)

Sau khi trích xuất được hai loại đặc trưng – **đặc trưng ảnh (2048 chiều)** và **đặc trưng câu hỏi (1024 chiều)** – mô hình tiến hành **kết hợp (fusion)** hai nguồn thông tin này để dự đoán câu trả lời.

Trong mã nguồn, hai vector đặc trưng được **nối trực tiếp theo chiều kênh** bằng phép `torch.cat`, tạo thành một vector hợp nhất có kích thước **3072 chiều** cho mỗi mẫu dữ liệu ($2048 + 1024$).

Tiếp đó, vector này được đưa qua **mạng MLP (Multilayer Perceptron)** gồm ba tầng tuyến tính:

1. Tầng đầu tiên: từ 3072 $\rightarrow$ 1024 nút ẩn,
2. Tầng thứ hai: từ 1024 $\rightarrow$ 512 nút ẩn,
3. Tầng cuối: từ 512 $\rightarrow$ số lượng nhãn đáp án (tối đa 1000).

Giữa các tầng có **hàm kích hoạt ReLU** và **Dropout** nhằm giảm overfitting. Đầu ra cuối cùng của mạng là một vector **logits** kích thước `[batch_size, num_answers]`, biểu diễn xác suất của từng câu trả lời có thể xảy ra.

Trong quá trình huấn luyện, mô hình sử dụng **hàm mất mát CrossEntropyLoss**, so sánh nhãn dự đoán với nhãn thật (`answer_tensor`). Câu trả lời trong tập dữ liệu được biểu diễn dưới dạng chỉ số, ứng với một từ duy nhất trong **từ vựng đáp án (answer vocabulary)**.

Do đó, đầu ra của mô hình là **một từ duy nhất** đại diện cho câu trả lời đúng nhất mà mô hình dự đoán.

```
class VQAModel(nn.Module):
    def __init__(self, question_vocab_size, answer_vocab_size, image_feature_dim=2048, question_hidden_dim=512, fusion_hidden_dim=1024, dropout=0.5):
        super().__init__()
        self.question_encoder = QuestionEncoder(question_vocab_size, hidden_dim=question_hidden_dim)
        self.image_encoder = ImageEncoder(image_feature_dim, pretrained=True)
        self.fusion_dim = image_feature_dim + question_hidden_dim * 2 # 2048 + 1024 = 3072
        self.fusion = nn.Sequential(
            nn.Linear(self.fusion_dim, fusion_hidden_dim), #3072 --> 1024
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(fusion_hidden_dim, fusion_hidden_dim // 2), #1024 --> 512
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(fusion_hidden_dim // 2, answer_vocab_size) #512 --> answer_vocab_size
        )

    def forward(self, image, question):
        f_img = self.image_encoder(image)
        h_q = self.question_encoder(question)
        fused = torch.cat((f_img, h_q), dim=1)
        output = self.fusion(fused)
        return output
```

2.3 Huấn luyện mô hình baseline và báo cáo kết quả (accuracy hoặc BLEU/ROUGE nếu là câu trả lời dài)

1. Mục tiêu

Mô hình baseline được xây dựng nhằm tạo ra một hệ thống cơ bản có khả năng trả lời câu hỏi dựa trên hình ảnh. Đây là mô hình nền tảng giúp đánh giá hiệu quả của các phương pháp nâng cao trong các bước nghiên cứu tiếp theo.

2. Thiết lập dữ liệu huấn luyện

Dữ liệu huấn luyện và kiểm thử được nạp thông qua DataLoader của PyTorch. Dữ liệu được chia thành các batch có kích thước 512 mẫu. Việc sử dụng shuffle ở tập huấn luyện giúp mô hình học ngẫu nhiên hơn, còn ở tập kiểm thử không shuffle để đảm bảo đánh giá ổn định.

3. Cấu hình mô hình

Mô hình baseline được định nghĩa thông qua lớp VQAModel. Trong đó, kích thước tập từ vựng của câu hỏi và câu trả lời được xác định để tạo đầu vào và đầu ra tương ứng. Mô hình gồm ba phần chính: mạng CNN dùng để trích xuất đặc trưng hình ảnh, mạng LSTM hoặc GRU dùng để mã hóa câu hỏi, và lớp fully connected dùng để kết hợp đặc trưng hình ảnh và câu hỏi trước khi dự đoán câu trả lời.

4. Cấu hình huấn luyện

Hàm mất mát sử dụng là CrossEntropyLoss, phù hợp cho bài toán phân loại nhiều lớp. Bộ tối ưu hóa Adam được dùng để cập nhật trọng số nhanh và ổn định. Mô hình được huấn luyện trong 40 epoch.

5. Quá trình huấn luyện

Trong mỗi epoch, dữ liệu được chia thành từng batch gồm hình ảnh, câu hỏi và câu trả lời. Mô hình thực hiện lan truyền thuận để dự đoán câu trả lời, tính toán sai số giữa dự đoán và nhãn thật, sau đó lan truyền ngược để cập nhật trọng số. Sau mỗi epoch, tính toán độ chính xác để theo dõi hiệu quả huấn luyện.

6. Giai đoạn đánh giá (Validation)

Cứ sau hai epoch, mô hình được chuyển sang chế độ đánh giá. Trong giai đoạn này, mô hình không cập nhật trọng số mà chỉ tính toán sai số và độ chính xác trên tập kiểm thử để đánh giá khả năng tổng quát hóa.

```
Epoch 1/40, Loss: 5.7643, Accuracy: 2.64%  
Val Loss: 4.8608, Val Accuracy: 4.85%  
Epoch 2/40, Loss: 5.0861, Accuracy: 5.98%  
Epoch 3/40, Loss: 4.7816, Accuracy: 8.66%  
Val Loss: 4.4790, Val Accuracy: 10.59%  
Epoch 4/40, Loss: 4.5456, Accuracy: 11.04%  
Epoch 5/40, Loss: 4.2837, Accuracy: 14.56%  
Val Loss: 3.9827, Val Accuracy: 18.04%  
Epoch 6/40, Loss: 4.0037, Accuracy: 17.92%  
Epoch 7/40, Loss: 3.7894, Accuracy: 20.64%  
Val Loss: 3.8113, Val Accuracy: 19.17%  
Epoch 8/40, Loss: 3.5758, Accuracy: 22.66%  
Epoch 9/40, Loss: 3.3799, Accuracy: 25.98%  
Val Loss: 3.7752, Val Accuracy: 19.33%  
Epoch 10/40, Loss: 3.1950, Accuracy: 28.83%  
Epoch 11/40, Loss: 3.0493, Accuracy: 30.26%  
Val Loss: 3.7345, Val Accuracy: 21.53%  
Epoch 12/40, Loss: 2.8532, Accuracy: 33.68%  
Epoch 13/40, Loss: 2.7082, Accuracy: 35.87%  
Val Loss: 3.7579, Val Accuracy: 22.05%  
Epoch 14/40, Loss: 2.5607, Accuracy: 37.20%  
Epoch 15/40, Loss: 2.4292, Accuracy: 39.08%  
Val Loss: 3.8629, Val Accuracy: 24.30%  
Epoch 16/40, Loss: 2.2924, Accuracy: 41.57%  
Epoch 17/40, Loss: 2.1494, Accuracy: 44.09%  
...  
Epoch 39/40, Loss: 0.8164, Accuracy: 73.94%  
Val Loss: 5.6503, Val Accuracy: 25.38%  
Epoch 40/40, Loss: 0.7913, Accuracy: 74.74%  
Fusion model saved!
```

```

Epoch 1/40, Loss: 5.6893, Accuracy: 6.10%
Val Loss: 5.0817, Val Accuracy: 4.40%
Epoch 2/40, Loss: 5.1174, Accuracy: 10.19%
Epoch 3/40, Loss: 4.7256, Accuracy: 13.08%
Val Loss: 4.3341, Val Accuracy: 19.85%
Epoch 4/40, Loss: 4.4039, Accuracy: 15.60%
Epoch 5/40, Loss: 4.1138, Accuracy: 17.39%
Val Loss: 3.8973, Val Accuracy: 22.26%
Epoch 6/40, Loss: 3.8737, Accuracy: 19.86%
Epoch 7/40, Loss: 3.6626, Accuracy: 21.96%
Val Loss: 3.6468, Val Accuracy: 23.65%
Epoch 8/40, Loss: 3.4354, Accuracy: 24.01%
Epoch 9/40, Loss: 3.1936, Accuracy: 26.55%
Val Loss: 3.4276, Val Accuracy: 24.21%
Epoch 10/40, Loss: 2.9993, Accuracy: 29.56%
Epoch 11/40, Loss: 2.7748, Accuracy: 32.29%
Val Loss: 3.2618, Val Accuracy: 28.51%
Epoch 12/40, Loss: 2.6081, Accuracy: 34.97%
Epoch 13/40, Loss: 2.4358, Accuracy: 38.28%
Val Loss: 3.1652, Val Accuracy: 31.69%
Epoch 14/40, Loss: 2.3069, Accuracy: 40.14%
Epoch 15/40, Loss: 2.1594, Accuracy: 43.06%
Val Loss: 3.1500, Val Accuracy: 33.61%
Epoch 16/40, Loss: 2.0513, Accuracy: 45.02%
Epoch 17/40, Loss: 1.9337, Accuracy: 47.04%
Val Loss: 3.2411, Val Accuracy: 34.17%
Epoch 18/40, Loss: 1.8442, Accuracy: 49.09%
Epoch 19/40, Loss: 1.7521, Accuracy: 51.10%
Val Loss: 3.2379, Val Accuracy: 35.71%
Epoch 20/40, Loss: 1.6744, Accuracy: 52.76%
Epoch 21/40, Loss: 1.5884, Accuracy: 54.13%
Val Loss: 3.3636, Val Accuracy: 36.65%
Epoch 22/40, Loss: 1.5485, Accuracy: 55.01%
Epoch 23/40, Loss: 1.4509, Accuracy: 57.25%
Val Loss: 3.4510, Val Accuracy: 36.62%
Epoch 24/40, Loss: 1.4150, Accuracy: 57.60%
Epoch 25/40, Loss: 1.3380, Accuracy: 60.68%
Val Loss: 3.5227, Val Accuracy: 36.90%
Epoch 26/40, Loss: 1.2937, Accuracy: 61.16%
Epoch 27/40, Loss: 1.2506, Accuracy: 62.76%
Val Loss: 3.6411, Val Accuracy: 36.72%
Epoch 28/40, Loss: 1.1746, Accuracy: 63.67%
Epoch 29/40, Loss: 1.1467, Accuracy: 65.75%
Val Loss: 3.7532, Val Accuracy: 37.60%
Epoch 30/40, Loss: 1.1036, Accuracy: 66.23%
Epoch 31/40, Loss: 1.0652, Accuracy: 67.30%
Val Loss: 3.7564, Val Accuracy: 38.85%
Epoch 32/40, Loss: 1.0553, Accuracy: 67.09%
Epoch 33/40, Loss: 0.9973, Accuracy: 68.92%

```

7. Lưu mô hình

Sau khi huấn luyện xong, mô hình được lưu lại thành tệp **vqa_fusion_model.pth** để phục vụ cho quá trình kiểm thử và ứng dụng sau này.

8. Kết luận

Mô hình baseline này thể hiện quy trình huấn luyện cơ bản của bài toán Visual Question Answering. Nó đảm bảo đầy đủ các bước từ xử lý dữ liệu, xây dựng mô hình, huấn luyện, đánh giá và lưu kết quả. Mô hình này đóng vai trò là nền tảng chuẩn để so sánh, đánh giá với các mô hình cải tiến trong các nghiên cứu tiếp theo.

```
batch_size = 256
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

# Model
question_vocab_size = len(train_dataset.question_vocab)
answer_vocab_size = len(train_dataset.answer_vocab)
model = VQAModel(question_vocab_size, answer_vocab_size).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

num_epochs = 40
for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    for batch in train_loader:
        images = batch['image_path'].to(device)
        questions = batch['question'].to(device)
        answers = batch['answer'].to(device)

        optimizer.zero_grad()
        outputs = model(images, questions)
        loss = criterion(outputs, answers)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += answers.size(0)
        correct += (predicted == answers).sum().item()

    avg_loss = total_loss / len(train_loader)
    accuracy = 100 * correct / total
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}, Accuracy: {accuracy:.2f}%")
```



```

# Val
if epoch % 2 == 0:
    model.eval()
    val_loss = 0
    val_correct = 0
    val_total = 0
    with torch.no_grad():
        for batch in val_loader:
            images = batch['image_path'].to(device)
            questions = batch['question'].to(device)
            answers = batch['answer'].to(device)
            outputs = model(images, questions)
            val_loss += criterion(outputs, answers).item()
            _, predicted = torch.max(outputs, 1)
            val_total += answers.size(0)
            val_correct += (predicted == answers).sum().item()
        print(f"Val Loss: {val_loss / len(val_loader):.4f}, Val Accuracy: {100 * val_correct / val_total:.2f}%")

# Save
torch.save(model.state_dict(), 'vqa_fusion_model.pth')
print("Fusion model saved!")

```

CHAPTER 3. *Đánh giá & báo cáo:*

3.1 Tiêu chí đánh giá:

Đánh giá dựa trên tập validation

```

model.eval()
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
total_correct = 0
total_samples = 0

with torch.no_grad():
    for batch in val_loader:
        images = batch['image_path'].to(device)
        questions = batch['question'].to(device)
        answers = batch['answer'].to(device)

        outputs = model(images, questions)
        predicted = outputs.argmax(dim=1)

        total_correct += (predicted == answers).sum().item()
        total_samples += answers.size(0)

# Calculate Top-1 Accuracy
top1_accuracy = (total_correct / total_samples) * 100 if total_samples > 0 else 0
print(f"Top-1 Accuracy on Validation Set: {top1_accuracy:.2f}% ({total_correct}/{total_samples} correct)")

```


Top-1 Accuracy on Validation Set: 39.73% (1137/2862 correct)

Qualitative examples:

Trả lời đúng:

Sample 23 - Image ID: image1227



what is in front of the sofa ? - table

Sample 26 - Image ID: image300



what is the colour of chair ? – black

Trả lời chưa đúng:

Sample 30 - Image ID: image43



what are objects on the top most shelves close to the wall ? – book

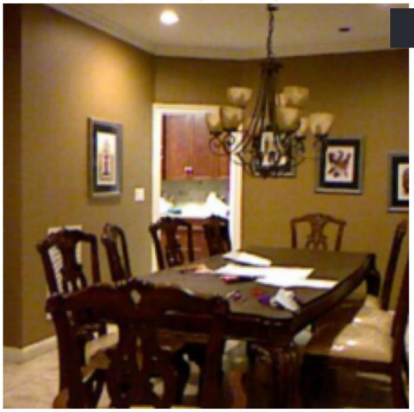
3.2 Bảng đánh giá hiệu suất và sample

Bảng đánh giá hiệu suất

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)	Gap (Train - Val) (%)
5	4.28	14.56	3.98	18.04	-3.48
10	3.20	28.83	3.73	21.53	7.30
15	2.43	39.08	3.86	24.30	14.78
20	1.82	49.43	4.14	24.02	25.41
25	1.41	58.04	4.51	23.98	34.06
30	1.13	65.76	4.79	24.26	41.50
35	0.93	70.50	3.75	37.60	32.90

Sample:

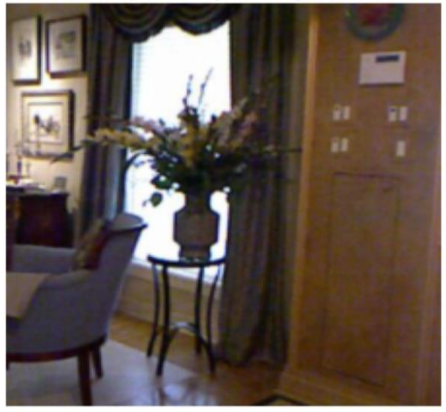
Sample 1 - Image ID: image1397



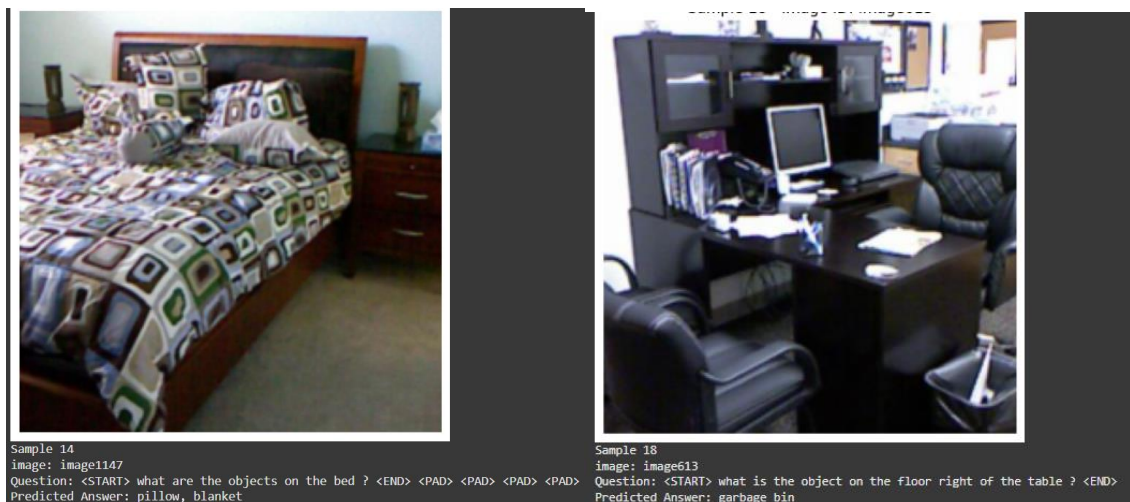
COLLAB.RESEARCH.C

Sample 1
Image: image1397
Question: <START> how many frames are there on the wall ? <END> <PAD> <PAD> <PAD>
Predicted Answer: 3
True Answer: 3

Sample 2 - Image ID: image1388



Sample 2
Image: image1388
Question: <START> what is to the left of the cardboard ? <END> <PAD> <PAD> <PAD>
Predicted Answer: cabinet
True Answer: window



3.3 Điểm mạnh, yếu và hướng phát triển

Điểm mạnh

Hiệu quả trong feature extraction:

- ImageEncoder (ResNet50 pre-trained, unfreeze layer4) trích xuất features cao cấp tốt, giúp model học nhanh ở simple questions
- QuestionEncoder (bi-LSTM) capture sequence tốt, đặc biệt với bidirectional, giúp xử lý câu hỏi dài (e.g., "what are objects on the top most shelves?").

Fusion đơn giản nhưng hiệu quả:

MultiheadAttention (seq_len=1) kết hợp image-question features mượt mà, giảm complexity so với full seq-seq attention. Dropout=0.5 và Xavier init giúp ổn định training.

Điểm yếu

Overfitting và generalization kém:

Train acc cao (69%) nhưng val chỉ ~39%, gap ~30% → Model memorize train data (imbalance vẫn tồn tại, rare answers dễ overfit sau oversample). Không có regularization mạnh (e.g., chỉ dropout cơ bản).

□ Vocab và classification hạn chế:

Answer vocab ~1000

□ Data và training constraints:

Dataset nhỏ (~11k train)

Hướng cải tiến

REFERENCES

Pattern Recognition and Machine Learning by Christopher M. Bishop Reference: Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville Reference: Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Machine Learning: A Probabilistic Perspective by Kevin P. Murphy Reference: Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron Reference: Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.

Machine Learning Yearning by Andrew Ng Reference: Ng, A. (2018). *Machine Learning Yearning*. Self-published.

Introduction to Machine Learning with Python: A Guide for Data Scientists by Andreas C. Müller and Sarah Guido Reference: Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media.

The Elements of Statistical Learning by Trevor Hastie, Robert Tibshirani, and Jerome Friedman Reference: Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.

Deep Reinforcement Learning Hands-On by Maxim Lapan Reference: Lapan, M. (2018). *Deep Reinforcement Learning Hands-On*. Packt Publishing.

Python Machine Learning by Sebastian Raschka and Vahid Mirjalili Reference: Raschka, S., & Mirjalili, V. (2019). *Python Machine Learning*. Packt Publishing.

Data Mining: Practical Machine Learning Tools and Techniques by Ian H. Witten, Eibe Frank, Mark A. Hall Reference: Witten, I. H., Frank, E., & Hall, M. A. (2016).

Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann.